Terraform

Job 01 - Préparation de l'environnement

L'objectif de cette phase est de **préparer l'environnement de travail nécessaire pour déployer automatiquement des machines virtuelles Debian 12** avec Terraform. Cette étape repose sur la création manuelle d'une machine virtuelle Debian 12 "modèle", configurée de manière minimale, mais suffisante pour permettre l'automatisation par la suite.

Étapes détaillées

1. Créer une VM Debian 12 sous VMware Workstation

Configuration minimale requise pour la VM :

- **Pas d'interface graphique** : choisir "installation serveur" uniquement.
- **Serveur SSH installé** : pour permettre l'accès distant.
- **Open-VM-Tools installés** : pour assurer l'intégration VMware (IP, synchronisation temps, etc.).
- **Utilisateur standard avec `sudo`** : pour exécuter des commandes administratives à distance.

Exemple de configuration pendant l'installation :

- Nom d'hôte : `debian12-Template`
- Partitionnement : automatique, disque entier
- Paquets à installer : `SSH server`, `standard system utilities`
- Créer un utilisateur : `terraform` avec mot de passe simple

Installer les outils VMware :

```
sudo apt update
sudo apt install open-vm-tools -y
```

2. Vérifier l'accès SSH

Assurer que la VM est bien accessible en SSH (depuis la machine hôte).

Exemple de test SSH :

```
ssh terraform@192.168.220.100
```

3. Noter le chemin complet vers le fichier `.vmx`

Ce chemin représente **le modèle (template)** à partir duquel Terraform va cloner les VMs.

Exemple de chemin :

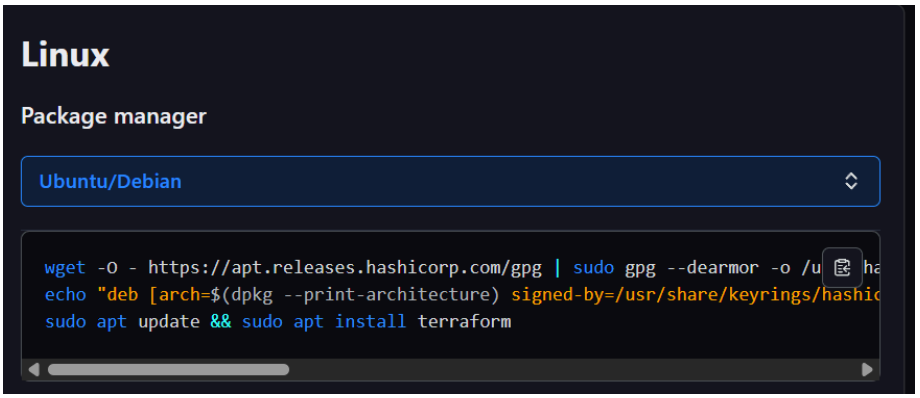
- Sous Windows : `C:\VMs\Debian12-Base\Debian12-Base.vmx`
- Sous Linux : `/home/user/vmware/Debian12-Base/Debian12-Base.vmx`

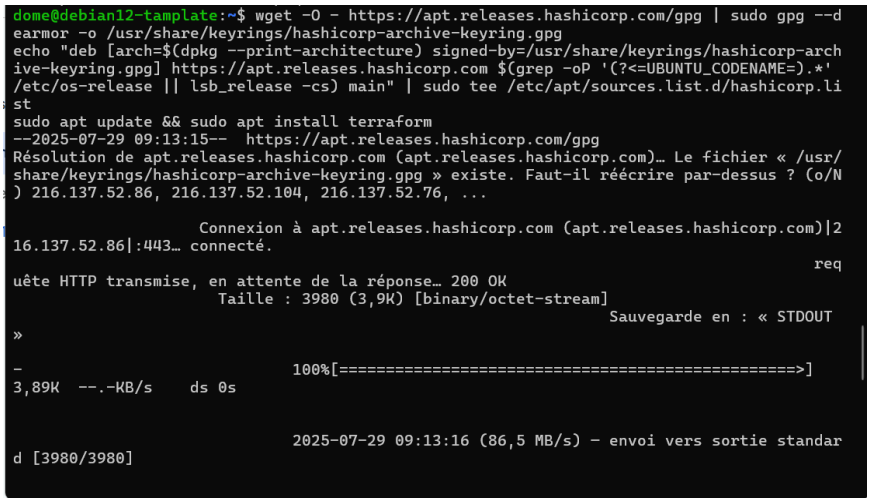
⚠️ Ce chemin sera utilisé dans le fichier `main.tf` plus tard.

Job 02- Installer Terraform

➤ Sous Linux (Debian/Ubuntu)

Récupérer les commandes du site officiel :
https://developer.hashicorp.com/terraform/install





Commande test :

```
terraform -v
```

Créer le répertoire de travail Terraform

```
mkdir ~/terraform_debian_lab
```

```
cd ~/terraform_debian_lab
```

```
dome@debian12-tamplate:~$ terraform -v
Terraform v1.12.2
on linux_amd64
dome@debian12-tamplate:~$ mkdir ~/terraform_debian_lab
cd ~/terraform_debian_lab
dome@debian12-tamplate:~/terraform_debian_lab$
```

- Premier déploiement de VM Debian 12

Job 03

Créer le fichier `main.tf`

Dans ce fichier, on va :

- Déclarer le provider `vmworkstation`
- Définir une ressource VM clonée depuis le template

Créer un fichier `main.tf` avec le contenu suivant :

```
terraform {
  required_providers {
    vmworkstation = {
      source = "elsudano/vmworkstation"
      version = "1.1.1"
    }
  }
}

provider "vmworkstation" {
  vmx_path = "D:\Program Files\VMware\VMware Player\Virtual Machines\Terraform-Master.vmx" # à adapter !
}

resource "vmworkstation_vm_clone" "debian_vm" {
  name        = "debian12-test"
  clone_from  = "/home/dome/vmware/Debian12-Base/Debian12-Base.vmx" # même chemin
  power_on    = true
  clone_path  = "/home/dome/vmware/debian12-test" # nouveau dossier VM clonée
}
```

Adapter les chemins selon votre environnement !

Initialiser Terraform

Dans ton dossier `terraform_debian_lab` :

```
terraform init
```

Cette commande :

- Télécharge le provider nécessaire
- Prépare l'environnement Terraform

```
dome@debian12-tamplate:~/terraform_debian_lab$ sudo nano main.tf
dome@debian12-tamplate:~/terraform_debian_lab$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of elsudano/vmworkstation...
- Installing elsudano/vmworkstation v2.0.1...
- Installed elsudano/vmworkstation v2.0.1 (self-signed, key ID 25B4C7DDA0B6F683)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
dome@debian12-tamplate:~/terraform_debian_lab$
```

cd "D:\Program Files (x86)\VMware\VMware Workstation" # ou le dossier réel

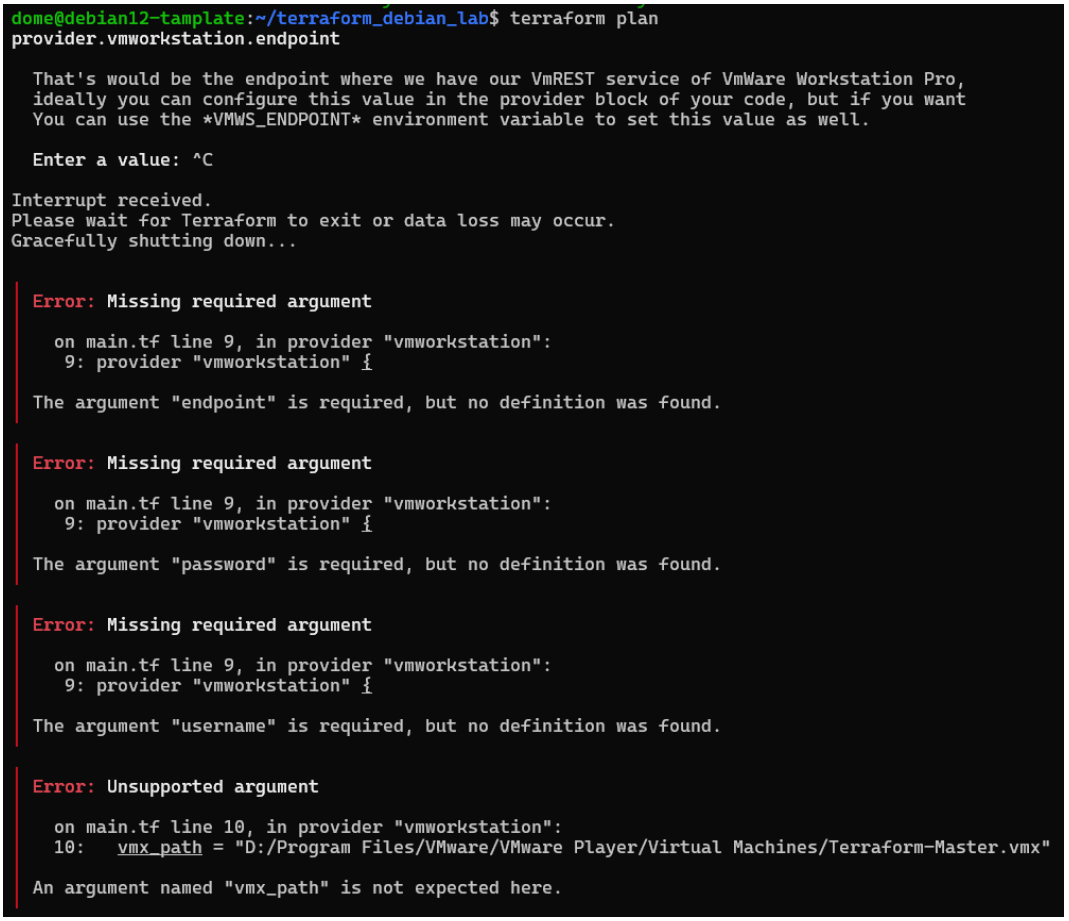
.\vmrest.exe -C

saisir un user/mot de passe quand demandé

Vérifier le plan d’exécution

`terraform plan`

Cette commande te montre ce que Terraform **va faire**, sans encore appliquer les changements.



Lancer le déploiement

```
terraform apply
```

```
dome@debian12-tamplate:~/terraform_debian_lab$ terraform apply
provider.vmworkstation.endpoint

  That's would be the endpoint where we have our VmREST service of VmWare Workstation Pro,
  ideally you can configure this value in the provider block of your code, but if you want
  You can use the *VMWS_ENDPOINT* environment variable to set this value as well.

  Enter a value: ^C

Interrupt received.
Please wait for Terraform to exit or data loss may occur.
Gracefully shutting down...


Error: Missing required argument

  on main.tf line 9, in provider "vmworkstation":
   9: provider "vmworkstation" {

The argument "password" is required, but no definition was found.


Error: Missing required argument

  on main.tf line 9, in provider "vmworkstation":
   9: provider "vmworkstation" {

The argument "username" is required, but no definition was found.


Error: Missing required argument

  on main.tf line 9, in provider "vmworkstation":
   9: provider "vmworkstation" {

The argument "endpoint" is required, but no definition was found.


Error: Unsupported argument

  on main.tf line 10, in provider "vmworkstation":
  10:   vmx_path = "D:/Program Files/VMware/VMware Player/Virtual Machines/Terraform-Master.vmx"

An argument named "vmx_path" is not expected here.
```

Ace stade notre script ne contient pas le scredentiels pour ce connecter

(facultative) Supprimer la VM clonée

Quand tu veux supprimer la VM testée :

```
terraform destroy
```

Étapes pour activer l'API VmREST sur VMware Workstation Pro

Même si tu as Workstation Pro (licencié), l'API VmREST **ne démarre pas toujours automatiquement**. Voici comment la forcer.

Vérifier la présence du fichier vmrest.exe sur la machine hote

Aller dans le dossier (en adaptant selon ton installation) :

```
D:\Program Files\VMware\VMware Player\Virtual Machines\Terraform-Maste
```

Trouver un fichier :

```
vmrest.exe
```

C'est le programme serveur qui expose l'API REST sur le port 8697.

Lancer vmrest.exe manuellement

Depuis **PowerShell en tant qu'administrateur**, exécuter :

```
cd "C:\Program Files (x86)\VMware\VMware Workstation"
```

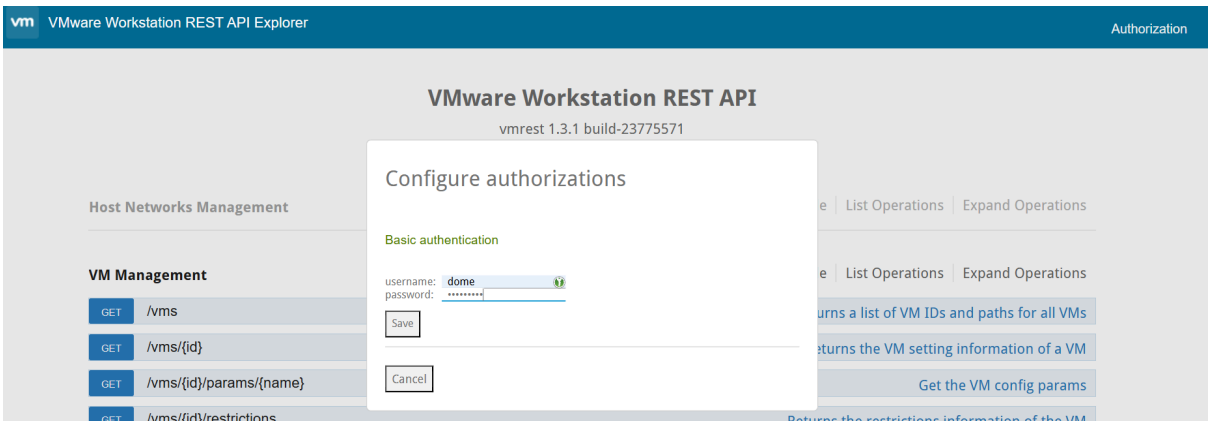
```
.\vmrest.exe
```

Si tout va bien, il va démarrer un service REST en local.

```
PS C:\WINDOWS\system32> cd "D:\Program Files\VMware\VMware Player"
PS D:\Program Files\VMware\VMware Player> .\vmrest.exe -C
VMware Workstation REST API
Copyright (C) 2018-2024 VMware Inc.
All Rights Reserved

vmrest 1.3.1 build-23775571
Username:dome
New password:
Password does not meet complexity requirements:
- Minimum 1 uppercase character
- Minimum 1 lowercase character
- Minimum 1 numeric digit
- Minimum 1 special character(!#$%&'()*+,-./:;<=>?@[]^_`{|}~)
- Length between 8 and 12
New password:
Retype new password:
Mismatch; try again, ctrl-c to quit.
New password:
Retype new password:
Mismatch; try again, ctrl-c to quit.
New password:
Retype new password:
Processing...
Credential updated successfully
PS D:\Program Files\VMware\VMware Player>
```

Un nouveau username et password devront être créés, ils seront indispensables pour s'identifier via la page web.



Job 04 - Amélioration et gestion du cycle de vie

Modifier ton fichier main.tf

Voici un **exemple corrigé** :

```
terraform {

  required_providers {

    vmworkstation = {

      source  = "elsudano/vmworkstation"

      version = "2.0.1"

    }

  }

}


provider "vmworkstation" {

  endpoint = var.vmws_endpoint

  username = var.vmws_username

  password = var.vmws_password

}


resource "vmworkstation_vm_clone" "debian_vm" {

  name        = "debian12-test"

  clone_from  = "D:/Program Files/VMware/VMware Player/Virtual
Machines/Terraform-Master/Terraform-Master.vmx"

  power_on    = true

  clone_path  = "D:/VMs/debian12-test"

}
```

Déclarer les variables

Ajoute un fichier variables.tf :

```
variable "vmws_endpoint" {
  description = "URL of the VmREST API"
  type        = string
}

variable "vmws_username" {
  description = "Username to connect to VmREST"
  type        = string
}

variable "vmws_password" {
  description = "Password to connect to VmREST"
  type        = string
}
```

Créer un fichier terraform.tfvars

Ce fichier contient les vraies valeurs **à ne jamais versionner sur GitHub** :

```
vmws_endpoint = "http://192.168.X.X:8697"
vmws_username = "your-windows-username"
vmws_password = "your-windows-password"
```

Remplacer bien l'IP (192.168.X.X) par l'IP de ta machine Windows exemple sur VMnet8.

Étapes finales

1. Sauvegarder tous les fichiers (main.tf, variables.tf, terraform.tfvars)
2. Refaire :

```
terraform init

terraform plan

terraform apply
```